

Constructor $\Rightarrow$ same name as class name $\Rightarrow$ (✓)

parameter $\Rightarrow$ method parameter $\Rightarrow$ (✓)

called ~~during~~ object created

(or) $\updownarrow$
instantiated

$\Rightarrow$ (✓)

return statement invalid $\Rightarrow$ (✓)

will not have return type explicitly $\Rightarrow$ (✓)

inside constructor first statement $\text{super}()$ (or) $\text{this}()$

$\text{super}()$ $\rightarrow$ calls $\rightarrow$ parent constructor

$\text{this}()$ $\rightarrow$ constructor of same class $\rightarrow$ (✓)

} constructor chaining

Both $\text{this}()$ & $\text{super}()$ in same constructor (X)

can write $\text{this}()$ (or) $\text{super}()$ $\Rightarrow$ apart from

first statement (X)

Encapsulation

FOR GETTER OF BOOLEAN WE DONT USE getVar rather we uses isVar

Keyword this

if java has one class we can create many objects from it

this keyword holds the address of the object currently is being executed

Constructor

constructor is a special method which is having same name of the class name and it doesnt have explicit return type whatever we can do in a method we can do to constructor too

Creation of constructor

```
class getData
{
    private int data;
    private String name;
    //creation of constructor;

    public getData(int data,String name)
    {
        this.data=data;
        this.name=name;
    }
}

public class encap {
    public static void main(String[] args) {
        getData gd=new getData(12,"Soha");
        // gd.data=12; error will be there as private keywords is used
        //we cant fetch it from outside of the class
    }
}
```

```

    }
}

```

- Construction is invoked at the time of instance creation
- IF THERE IS NOT CONSTRUCTOR CONSTRUCTED INSIDE CLASS JVM WILL AUTOMATICALLY INCLUDE A DEFAULT CONSTRUCTOR;
- The default constructor doesn't have any parameters;
- if we declare any of the even one constructor then jvm will not include the default constructor

constructor overloading

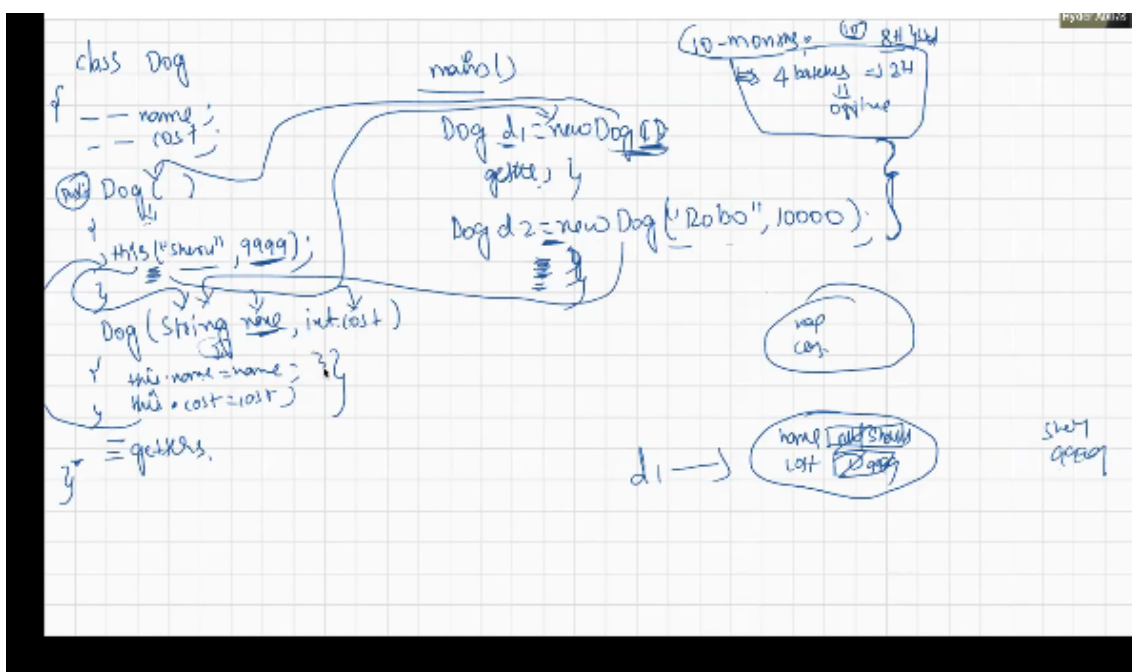
constructor overloading refers to the process of having multiple constructors having differences in parameters by several factors such as number of parameters, their data types and etc..

this() vs super()

Whenever any of the constructors is executed by default the `super()` method is executed behind the scene while executing. However, we can explicitly mention it too.

But if we use the `this()` method which is used to call the constructor of the same class, then `super()` won't be getting executed. We can call either of the above methods.

constructor chaining



method

constructor

⇒ ~~call~~ call explicitly by
calling name

⇒ return type explicitly
(void, int, string)

⇒ return statement is
allowed

⇒ when object is created
constructor is called.

⇒ no explicit return type
~~return~~

⇒ no return statement

this } this()
keyword } method()
↓
refer to current object } ⇒ it will call same class constructor

super

super()

} After Inheritance }

super() method ⇒ method

⇒ more than one constructor with diff parameter

↓

method overloading

contact details

haider : syedhyder@ineuron.ai
nitin : nitin@ineuron.ai

Given

```
1. class Dims {  
2.     public static void main(String[] args) {  
3.         int[][] a = {{1,2},{3,4}};  
4.         int[] b = (int[]) a[1];  
5.         Object o1 = a;  
6.         int[][] a2 = (int[][]) o1;  
7.         System.out.println(b[1]);  
8.     }  
9. }
```

What is the result?

- A. 2
- B. 4
- C. An exception is thrown at runtime
- D. Compilation fails due to an error on line 4.
- E. Compilation fails due to an error on line 5.
- F. Compilation fails due to an error on line 6.

Answer: B

Given

```
1. class Zippy {  
2.     String[] x;  
3.     int[] a [] = {{1,2}, {1}};  
4.     Object c = new long[4];  
5.     Object[] d = x;  
6. }
```

What is the result?

- A. Compilation succeeds.
- B. Compilation fails due only to an error on line 3.
- C. Compilation fails due only to an error on line 4.
- D. Compilation fails due only to an error on line 5.
- E. Compilation fails due to errors on lines 3 and 5.
- F. Compilation fails due to errors on lines 3, 4, and 5.

Answer: A

Q>

```
public class Fizz {  
    int x = 5;  
    public static void main(String[] args) {  
        final Fizz f1 = new Fizz();  
        Fizz f2 = new Fizz();  
        Fizz f3 = FizzSwitch(f1,f2);  
        System.out.println((f1 == f3) + " " + (f1.x == f3.x));  
    }  
    static Fizz FizzSwitch(Fizz x, Fizz y) {  
        final Fizz z = x;  
        z.x = 6;  
        return z;  
    }  
}
```

What is the result?

- A. true true

- B. false true
- C. true false
- D. false false
- E. Compilation fails.
- F. An exception is thrown at runtime.

Answer: A

Q>

Given

```
42. public class ClassA {
43.     public int getValue() {
44.         int value=0;
45.         boolean setting = true;
46.         String title="Hello";
47.         if (value || (setting && title == "Hello")) { return 1; }
48.         if (value == 1 & title.equals("Hello")) { return 2; }
49.     }
50. }
```

And:

```
70. ClassA a = new ClassA();
71. a.getValue();
```

What is the result?

- A. 1
- B. 2
- C. Compilation fails.
- D. The code runs with no output.
- E. An exception is thrown at runtime.

Answer: C (operand is not of boolean type is line no 47)

Q>

Given

```
int a= 8,b=15,c=4;
System.out.println( 2 * ((a%5) * (4+(b-3)/(c+2))));
What is the output?
```

- A. 30
- B. 36
- C. 32
- D. 35

evaluation

```
2 * ((8%5) * (4+(15-3)/(4+2)))
2 * ( (3      * ( 4 + 12/6)))
2 * ( (3      * ( 4 + 2))
2 * ( ( 3      * 6))
2 * (18)
36
```

answer: B

Q>

What will be the result of compiling and executing DivModTest class?

```
public class DivModTest {
    public static void main(String[] args) {
        System.out.println( 23 / 2.0 ); // 11.5
        System.out.println( 23 % 2.0 );// 1.0
    }
}
```

}

- A. 11
1
- B. 11.5
1.0
- C. 11.0
1.0
- D. 11.5
0.0

Answer: B

Q>

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello" + 1 + 2 + 3 + 4);  
    }  
}
```

- A. Hello10
- B. Hello19
- C. Hello1234
- D. Hello10

Answer: C

Q>

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(1 + 2 + 3 + 4 + "Hello");  
    }  
}
```

- A. 1234Hello
- B. 10Hello
- C. 64Hello
- D. 10 Hello

Answer: B(10Hello)

Q> What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Output is: " + 10 != 5);  
    }  
}
```

- A. Output is : true
- B. Output is : false
- C. Compilation error
- D. Output is : 10 !=5

Answer: C(On String Objects only + operator is allowed other operators would result in CompileTimeError)

Q>

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Output is: " + (10 != 5));  
    }  
}
```

```
    }  
}
```

- A. Output is : true
- B. Output is : false
- C. Compilation error
- D. Output is : 10 !=5

Answer: A

Q>

What will be the result of compiling and executing the Test class?

```
public class Test {  
    public static void main(String[] args) {  
        int grade = 75;  
        if(grade > 60)  
            System.out.println("Congratulations");  
            System.out.println("You passed");  
        else  
            System.out.println("You failed");  
    }  
}
```

- A. Congratulations
- B. Congratulations
You passed
- C. You failed
- D. compilation error

Answer: D

Q>

What will be the result of compiling and executing the Test class?

```
public class Test {  
    public static void main(String[] args) {  
        int grade = 60;  
        if(grade = 60)  
            System.out.println("You passed...");  
        else  
            System.out.println("You failed...");  
    }  
}
```

- A. You passed...
- B. You failed...
- C. compilation error
- D. Produces not output

Answer: C

Q>

Predict the following output for the given code

```
public class TestApp{  
    public static void main(String... args){  
        String[] arr[] ={{"%", "****"}, {"!!!!", "####", "#####"}};  
        for(String str[] : arr){  
            for(String s:str){  
                System.out.println(s);  
                if(s.length()==4)  
                    break;  
            }  
        }  
    }  
}
```

```

        break;
    }
}

```

A. Compile Time Error
 B. StringIndexOutOfBoundsException
 C. %

 D. %

 #####
 E. ArrayIndexOutOfBoundsException
 E. None of the above

Answer: C

Q>
What will be the result of compiling and executing Test class?

```

public class Test {
    public static void main(String[] args) {
        int score = 60;
        switch (score) {
            default:
                System.out.println("Not a valid score");
            case score < 70:
                System.out.println("Failed");
                break;
            case score >= 70:
                System.out.println("Passed");
                break;
        }
    }
}

```

A. Compilation Error
 B. Failed
 C. Not a valid score
 Failed
 D. Passed

Answer: A

Q>
What will be the result of compiling and executing Test class?

```

public class Test {
    public static void main(String[] args) {
        String fruit = "mango";
        switch (fruit) {
            case "Apple":
                System.out.println("APPLE");
            case "Mango":
                System.out.println("MANGO");
            case "Banana":
                System.out.println("BANANA");
                break;
            default:
                System.out.println("ANY FRUIT WILL DO");
        }
    }
}

```

- ```

 }
}
A. MANGO
B. ANY FRUIT WILL DO
C. MANGO
 BANNANA
D. MANGO
 ANY FRUIT WILL DO
E. MANGO
 BANNANA
 ANY FRUIT WILL DO

```

Answer: B

What will be the result of compiling and executing Test class?

```

public class Test {
 public static void main(String[] args) {
 String fruit = "mango";
 switch (fruit) {
 default:
 System.out.println("ANY FRUIT WILL DO");
 case "Apple":
 System.out.println("APPLE");
 case "Mango":
 System.out.println("MANGO");
 case "Banana":
 System.out.println("BANANA");
 break;
 }
 }
}

```

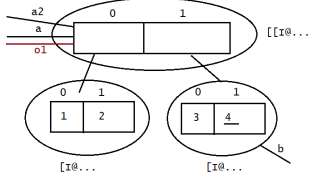
- ```

}
A. ANY FRUIT WILL DO
B. MANGO
C. MANGO
  BANNANA
D. ANY FRUIT WILL DO
  APPLE
  MANGO
  BANNANA

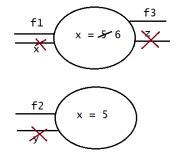
```

Answer: D

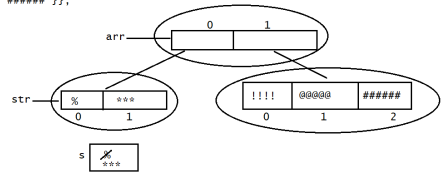

```
int[][] a = {{1,2}, {3,4}};
int[] b = (int[]) a[1];
Object o1 = a;
int[] a2 = (int[]) o1;
System.out.println(b[1]); //4
```



```
public class Fizz {
    int x = 5;
    public static void main(String[] args) {
        final Fizz f1 = new Fizz();
        Fizz f2 = new Fizz();
        Fizz f3 = Fizzswitch(f1, f2);
        System.out.println((f1 == f3) + " " + (f1.x == f3.x));
    }
    static Fizz Fizzswitch(Fizz x, Fizz y) {
        final Fizz z = x;
        z.x = 6;
        return z;
    }
}
```



```
String[] arr[] ={{"%", "***"}, {"!!!!", "####"}, {"#####"};
for(String str[] : arr){
    for(String s:str){
        System.out.println(s);
        if(s.length()==4)
            break;
    }
    break;
}
```



```
output
%
***
```